

EDUGENIE

Google Gemini 3.8 Flash Powered Learning Assistant

PHASE-WISE PROJECT DOCUMENTATION WITH COMPLETE SOURCE CODE

1. Project Overview

EduGenie is an AI-powered educational learning assistant built with Python, FastAPI, HTML, CSS, JavaScript, and Google Gemini 3.8 Flash. The implementation provides question answering, topic explanation, summarization, quiz generation, and personalized learning recommendations.

2. Objectives

Provide an accessible AI-based educational assistant; answer educational questions; explain topics at learner level; summarize text; generate structured quizzes; provide learning recommendations; keep credentials separate from source code; and provide API testing.

3. Technology Stack

Python, FastAPI, Google GenAI SDK, Gemini 3.8 Flash, Pydantic, Pydantic Settings, Jinja2, HTML/CSS/JavaScript, Pytest, HTTPX, and optional LaMini-Flan-T5-783M.

4. Project Structure

```
EduGenie/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── schemas.py
│   ├── gemini_service.py
│   ├── local_explainer.py
│   └── services.py
├── templates/index.html
├── static/style.css
├── static/app.js
├── tests/test_api.py
├── scripts/test_api.ps1
├── .env / .env.example
├── requirements.txt
└── README.md
```

Phase 1 – Project Setup & Configuration

- Create the project structure.
- Create a Python virtual environment.
- Install dependencies from requirements.txt.
- Configure .env using .env.example.
- Keep the Gemini credential out of source code and version control.

Implementation: requirements.txt, app/config.py

Phase 2 – FastAPI Backend & Data Validation

- Initialize FastAPI.
- Mount static files and Jinja2 templates.
- Create home and health endpoints.
- Expose Q&A, explanation, summarization, quiz, and recommendation endpoints.
- Validate request and response data with Pydantic.

Implementation: app/main.py, app/schemas.py

Phase 3 – Gemini AI & Local Explainer Integration

- Initialize Google GenAI client from environment configuration.
- Generate normal text responses.

- Generate schema-constrained JSON responses.
- Optionally use LaMini-Flan-T5 for local explanation with Gemini fallback.

Implementation: `app/gemini_service.py`, `app/local_explainer.py`

Phase 4 – Educational Features & Service Layer

- Implement Q&A; prompts.
- Implement learner-level topic explanations.
- Implement faithful summaries.
- Generate exactly three questions with four options and one correct answer each.
- Generate ordered learning paths.

Implementation: `app/services.py`

Phase 5 – Frontend Development

- Build the educational task selector.
- Collect learner level and goal.
- Send asynchronous JSON requests to FastAPI.
- Render text, quiz, and recommendation results.
- Provide loading, error, and responsive UI states.

Implementation: `templates/index.html`, `static/style.css`, `static/app.js`

Phase 6 – Testing & API Validation

- Test home page availability.
- Test health endpoint and configured model name.
- Test invalid Q&A;, explanation, and quiz input.
- Provide a PowerShell health-check script.

Implementation: `tests/test_api.py`, `scripts/test_api.ps1`

Phase 7 – Final Integration & Execution

- Run the FastAPI application locally.
- Open the web interface.
- Exercise all five learning tasks.
- Run pytest and the PowerShell health check.
- Verify configuration and API credentials are not exposed.

5. Implemented API Endpoints

Method	Endpoint	Purpose
GET	/	Render EduGenie web interface
GET	/health	Health check
POST	/qa	Answer educational question
POST	/explain	Explain a topic
POST	/summarize	Summarize text
POST	/quiz	Generate structured quiz
POST	/learn/recommendations	Generate learning path

6. Environment Configuration

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
GEMINI_MODEL=gemini-3.8-flash
GEMINI_THINKING_LEVEL=medium
APP_HOST=127.0.0.1
APP_PORT=8000
USE_LOCAL_EXPLAINER=false
LOCAL_MODEL_NAME=MBZUI/LaMini-Flan-T5-783M
```

The supplied .env contained a real-looking credential. It is intentionally omitted here and should be rotated/revoked if exposed.

7. Installation & Testing

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
pytest
.\scripts\test_api.ps1
```

8. Complete Source Code

app/main.py

```
from pathlib import Path

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

from .schemas import (
    QARequest, ExplainRequest, TextRequest, LearningPathRequest,
)

from .services import (
    answer_question, explain_topic, summarize, generate_quiz,
    learning_recommendations,
)

BASE_DIR = Path(__file__).resolve().parent.parent
STATIC_DIR = BASE_DIR / "static"
TEMPLATES_DIR = BASE_DIR / "templates"

app = FastAPI(
    title="EduGenie - Gemini 3.8 Flash",
    version="1.0.0",
    description="AI-powered educational assistant using Google Gemini 3.8 Flash.",
)

app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
templates = Jinja2Templates(directory=TEMPLATES_DIR)

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(request=request, name="index.html")

@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie", "model": "gemini-3.8-flash"}

@app.post("/qa")
async def qa(request: QARequest):
    try:
        answer = answer_question(request)
        return {"answer": answer}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/explain")
async def explain(request: ExplainRequest):
    try:
        explanation = explain_topic(request)
        return {"explanation": explanation}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/summarize")
```

```

async def summarize_text(request: TextRequest):
    try:
        summary = summarize(request)
        return {"summary": summary}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/quiz")
async def quiz(request: TextRequest):
    try:
        return generate_quiz(request)
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

@app.post("/learn/recommendations")
async def recommendations(request: LearningPathRequest):
    try:
        return learning_recommendations(request)
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc))

```

app/config.py

```

from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    gemini_api_key: str = ""
    google_api_key: str = ""
    gemini_model: str = "gemini-3.8-flash"
    gemini_thinking_level: str = "medium"
    app_host: str = "127.0.0.1"
    app_port: int = 8000
    use_local_explainer: bool = False
    local_model_name: str = "MBZUAI/LaMini-Flan-T5-783M"

    model_config = SettingsConfigDict(
        env_file=".env", env_file_encoding="utf-8",
        extra="ignore", case_sensitive=False,
    )

    @property
    def effective_api_key(self) -> str:
        return self.gemini_api_key or self.google_api_key

@lru_cache
def get_settings() -> Settings:
    return Settings()

```

app/schemas.py

```

from typing import List
from pydantic import BaseModel, Field, model_validator

class QARequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=10000)
    learner_level: str = "beginner"

class ExplainRequest(BaseModel):
    topic: str = Field(..., min_length=1, max_length=5000)
    learner_level: str = "beginner"

class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=30000)

class LearningPathRequest(BaseModel):
    topic: str = Field(..., min_length=1, max_length=5000)
    learner_level: str = "beginner"
    goal: str = "understand the fundamentals"

class QuizOption(BaseModel):
    text: str
    is_correct: bool

class QuizQuestion(BaseModel):
    question: str
    options: List[QuizOption]

    @model_validator(mode="after")
    def validate_options(self):
        if len(self.options) != 4:
            raise ValueError("Each quiz question must have exactly 4 options.")
        correct_answers = sum(option.is_correct for option in self.options)
        if correct_answers != 1:
            raise ValueError("Each quiz question must have exactly one correct option.")
        return self

class QuizResponse(BaseModel):
    questions: List[QuizQuestion]

    @model_validator(mode="after")
    def validate_questions(self):
        if len(self.questions) != 3:
            raise ValueError("The quiz must contain exactly 3 questions.")
        return self

class LearningStep(BaseModel):
    step: int
    topic: str
    description: str
    activity: str

class LearningPathResponse(BaseModel):
    recommendations: List[LearningStep]

```

app/gemini_service.py

```

from typing import Type, TypeVar

```

```

from google import genai
from google.genai import types
from pydantic import BaseModel
from .config import get_settings

T = TypeVar("T", bound=BaseModel)

class GeminiService:
    def __init__(self):
        settings = get_settings()
        if not settings.effective_api_key:
            raise RuntimeError(
                "Gemini API key is missing. Add GEMINI_API_KEY to your .env file."
            )
        self.settings = settings
        self.client = genai.Client(api_key=settings.effective_api_key)

    def generate_text(self, prompt: str) -> str:
        response = self.client.models.generate_content(
            model=self.settings.gemini_model,
            contents=prompt,
            config=types.GenerateContentConfig(
                thinking_config=types.ThinkingConfig(
                    thinking_level=self.settings.gemini_thinking_level
                )
            ),
        )
        if not response.text:
            raise RuntimeError("Gemini returned an empty response.")
        return response.text.strip()

    def generate_structured(self, prompt: str, schema: Type[T]) -> T:
        response = self.client.models.generate_content(
            model=self.settings.gemini_model,
            contents=prompt,
            config=types.GenerateContentConfig(
                response_mime_type="application/json",
                response_schema=schema,
                thinking_config=types.ThinkingConfig(
                    thinking_level=self.settings.gemini_thinking_level
                )
            ),
        )
        if not response.text:
            raise RuntimeError("Gemini returned an empty structured response.")
        return schema.model_validate_json(response.text)

def get_gemini_service() -> GeminiService:
    return GeminiService()

```

app/local_explainer.py

```

from functools import lru_cache

@lru_cache(maxsize=1)
def _get_pipeline():
    try:
        from transformers import pipeline
    except ImportError as exc:
        raise RuntimeError(
            "Local explainer is enabled, but transformers is not installed."
        ) from exc
    return pipeline(
        "text2text-generation",
        model="MBZUAI/LaMini-Flan-T5-783M",
        device=-1,
    )

def explain_with_local_model(topic: str, learner_level: str) -> str:
    generator = _get_pipeline()
    prompt = (
        f"Explain {topic} to a {learner_level} learner "
        "using simple language and examples."
    )
    result = generator(prompt, max_new_tokens=400)
    return result[0]["generated_text"].strip()

```

app/services.py

```

from .config import get_settings
from .gemini_service import get_gemini_service
from .local_explainer import explain_with_local_model
from .schemas import (
    QAResponse, ExplainRequest, TextRequest, LearningPathRequest,
    QuizResponse, LearningPathResponse,
)

def answer_question(request: QAResponse) -> str:
    gemini = get_gemini_service()
    prompt = f"""
You are EduGenie, an educational learning assistant.
Answer the following question accurately and clearly.
Learner level:
{request.learner_level}
Question:
{request.question}
Instructions:
- Explain using learner-friendly language.
- Give a simple example when useful.
- Break difficult concepts into smaller parts.
- Do not invent facts.
- Keep the answer educational and practical.
"""
    return gemini.generate_text(prompt)

def explain_topic(request: ExplainRequest) -> str:
    settings = get_settings()
    if settings.use_local_explainer:
        try:

```

```

        return explain_with_local_model(request.topic, request.learner_level)
    except Exception:
        pass
    gemini = get_gemini_service()
    prompt = f"""
You are EduGenie, an expert educational tutor.
Explain the following topic to a {request.learner_level} learner.
Topic:
{request.topic}
Structure the answer as:
1. Simple definition
2. Core idea
3. Step-by-step explanation
4. Practical example
5. Common mistakes
6. Short recap
Use clear educational language.
"""
    return gemini.generate_text(prompt)

def summarize(request: TextRequest) -> str:
    gemini = get_gemini_service()
    prompt = f"""
Summarize the following educational text.
Text:
{request.text}
Return:
1. A concise summary
2. Five key points
3. Important terms, if any
Keep the summary faithful to the original text.
Do not introduce unrelated information.
"""
    return gemini.generate_text(prompt)

def generate_quiz(request: TextRequest) -> QuizResponse:
    gemini = get_gemini_service()
    prompt = f"""
Create an educational quiz from the content below.
Content:
{request.text}
STRICT REQUIREMENTS:
- Exactly 3 questions.
- Exactly 4 options for every question.
- Exactly 1 correct option for every question.
- Each option must be relevant.
- Mark exactly one option with is_correct=true.
- Mark all other options with is_correct=false.
- Do not create more than 3 questions.
"""
    return gemini.generate_structured(prompt, QuizResponse)

def learning_recommendations(request: LearningPathRequest) -> LearningPathResponse:
    gemini = get_gemini_service()
    prompt = f"""
Create a personalized learning path.
Topic:
{request.topic}
Learner level:
{request.learner_level}
Learning goal:
{request.goal}
Create 5 to 7 ordered learning steps.
Every step must contain:
- step
- topic
- description
- activity
Start with prerequisites and gradually progress towards the learner's goal.
"""
    return gemini.generate_structured(prompt, LearningPathResponse)

```

static/app.js

```

const task = document.getElementById("task");
const inputText = document.getElementById("inputText");
const inputLabel = document.getElementById("inputLabel");
const levelGroup = document.getElementById("level-group");
const goalGroup = document.getElementById("goal-group");
const goal = document.getElementById("goal");
const submitBtn = document.getElementById("submitBtn");
const result = document.getElementById("result");
const loading = document.getElementById("loading");

function updateForm() {
    const value = task.value;
    levelGroup.classList.toggle("hidden",
        value !== "qa" && value !== "explain" && value !== "recommendations");
    goalGroup.classList.toggle("hidden", value !== "recommendations");
    if (value === "qa") {
        inputLabel.textContent = "Your Question";
        inputText.placeholder = "Example: Explain photosynthesis in simple terms.";
    } else if (value === "explain") {
        inputLabel.textContent = "Topic";
        inputText.placeholder = "Example: Newton's laws of motion";
    } else if (value === "summarize") {
        inputLabel.textContent = "Text to Summarize";
        inputText.placeholder = "Paste the text you want summarized...";
    } else if (value === "quiz") {
        inputLabel.textContent = "Educational Content";
        inputText.placeholder = "Paste your notes or educational content...";
    } else {
        inputLabel.textContent = "Topic";
        inputText.placeholder = "Example: Python programming";
    }
}

function escapeHtml(value) {
    return String(value).replaceAll("&", "&amp;").replaceAll("<", "&lt;");
}

```

```

        .replaceAll(">", "&gt;").replaceAll("'", "&quot;").replaceAll('"', "&#039;");
    }

    function renderQuiz(data) {
        return data.questions.map((question, index) => `
            <div class="quiz-question">
                <h3>${index + 1}. ${escapeHtml(question.question)}</h3>
                ${question.options.map((option, optionIndex) => `
                    <div class="option ${option.is_correct ? "correct" : ""}">
                        ${String.fromCharCode(65 + optionIndex)}.
                        ${escapeHtml(option.text)}
                        ${option.is_correct ? " ✓ Correct" : ""}
                    </div>`).join("")}
                </div>`).join("");
    }

    function renderRecommendations(data) {
        return data.recommendations.map((item) => `
            <div class="step">
                <h3>Step ${item.step}: ${escapeHtml(item.topic)}</h3>
                <p>${escapeHtml(item.description)}</p>
                <strong>Activity:</strong> ${escapeHtml(item.activity)}
            </div>`).join("");
    }

    task.addEventListener("change", updateForm);

    submitBtn.addEventListener("click", async () => {
        const value = task.value;
        const text = inputText.value.trim();
        if (!text) {
            result.innerHTML = "<p>Please enter some content first.</p>";
            return;
        }
        let endpoint;
        let body;
        if (value === "qa") {
            endpoint = "/qa";
            body = {question: text, learner_level: document.getElementById("level").value};
        } else if (value === "explain") {
            endpoint = "/explain";
            body = {topic: text, learner_level: document.getElementById("level").value};
        } else if (value === "summarize") {
            endpoint = "/summarize";
            body = {text: text};
        } else if (value === "quiz") {
            endpoint = "/quiz";
            body = {text: text};
        } else {
            endpoint = "/learn/recommendations";
            body = {
                topic: text,
                learner_level: document.getElementById("level").value,
                goal: goal.value.trim() || "understand the fundamentals"
            };
        }
        loading.classList.remove("hidden");
        result.innerHTML = "";
        submitBtn.disabled = true;
        try {
            const response = await fetch(endpoint, {
                method: "POST",
                headers: {"Content-Type": "application/json"},
                body: JSON.stringify(body)
            });
            const data = await response.json();
            if (!response.ok) throw new Error(data.detail || "Request failed.");
            if (value === "qa") result.innerHTML = `<div class="text-result">${escapeHtml(data.answer)}</div>`;
            else if (value === "explain") result.innerHTML = `<div class="text-result">${escapeHtml(data.explanation)}</div>`;
            else if (value === "summarize") result.innerHTML = `<div class="text-result">${escapeHtml(data.summary)}</div>`;
            else if (value === "quiz") result.innerHTML = renderQuiz(data);
            else result.innerHTML = renderRecommendations(data);
        } catch (error) {
            result.innerHTML = `<div class="error">${escapeHtml(error.message)}</div>`;
        } finally {
            loading.classList.add("hidden");
            submitBtn.disabled = false;
        }
    });
    updateForm();

```

static/style.css

```

* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, Helvetica, sans-serif; background: #f4f7fb; color: #172033; }
.container { width: min(950px, 92%); margin: auto; padding: 40px 0 60px; }
.hero { text-align: center; margin-bottom: 28px; }
.hero h1 { font-size: 48px; margin: 12px 0; }
.hero p { max-width: 720px; margin: auto; color: #5b6475; line-height: 1.6; }
.badge { display: inline-block; padding: 7px 12px; border-radius: 999px; background: #e7ecff; font-weight: bold; font-size: 13px; }
.card { background: white; border-radius: 16px; padding: 25px; margin-bottom: 22px; box-shadow: 0 8px 30px rgba(0, 0, 0, 0.07); }
label { display: block; font-weight: bold; margin: 14px 0 7px; }
input, select, textarea { width: 100%; padding: 12px; border: 1px solid #d5dae5; border-radius: 9px; font: inherit; }
textarea { resize: vertical; }
button { margin-top: 18px; width: 100%; border: 0; padding: 13px; border-radius: 9px; font-weight: bold; cursor: pointer; background: #3f51b5; color: white; }
button:disabled { opacity: 0.6; cursor: not-allowed; }
.hidden { display: none; }
.text-result { white-space: pre-wrap; line-height: 1.7; }
.quiz-question, .step { padding: 16px; border: 1px solid #e0e4ed; border-radius: 10px; margin-bottom: 15px; }
.option { padding: 11px; border-radius: 7px; background: #f5f6f9; margin: 8px 0; }
.option.correct { background: #e8f6ec; }
.error { color: #a32020; background: #fff0f0; padding: 14px; border-radius: 8px; }
#loading { margin-bottom: 12px; font-weight: bold; }
@media (max-width: 600px) {
    .container { padding-top: 20px; }
    .hero h1 { font-size: 38px; }
    .card { padding: 18px; }
}

```

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduGenie - Gemini 3.8 Flash</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
<main class="container">
  <section class="hero">
    <span class="badge">Gemini 3.8 Flash</span>
    <h1>EduGenie</h1>
    <p>Your AI-powered learning assistant for questions, explanations, quizzes, summaries, and personalized learning paths.</p>
  </section>
  <section class="card">
    <label for="task">Choose a task</label>
    <select id="task">
      <option value="qa">Ask a Question</option>
      <option value="explain">Explain a Topic</option>
      <option value="summarize">Summarize Text</option>
      <option value="quiz">Generate Quiz</option>
      <option value="recommendations">Learning Recommendations</option>
    </select>
    <div id="level-group">
      <label for="level">Learner Level</label>
      <select id="level">
        <option value="beginner">Beginner</option>
        <option value="intermediate">Intermediate</option>
        <option value="advanced">Advanced</option>
      </select>
    </div>
    <div id="goal-group" class="hidden">
      <label for="goal">Learning Goal</label>
      <input id="goal" type="text" placeholder="Example: prepare for an exam">
    </div>
    <label for="inputText" id="inputLabel">Your Question</label>
    <textarea id="inputText" rows="9" placeholder="Type your question here..."></textarea>
    <button id="submitBtn">Generate</button>
  </section>
  <section class="card">
    <h2>Result</h2>
    <div id="loading" class="hidden">EduGenie is thinking...</div>
    <div id="result">Your result will appear here.</div>
  </section>
</main>
<script src="/static/app.js"></script>
</body>
</html>
```

tests/test_api.py

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert "EduGenie" in response.text

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["model"] == "gemini-3.8-flash"

def test_qa_validation():
    response = client.post("/qa", json={"question": ""})
    assert response.status_code == 422

def test_explain_validation():
    response = client.post("/explain", json={"topic": ""})
    assert response.status_code == 422

def test_quiz_validation():
    response = client.post("/quiz", json={"text": ""})
    assert response.status_code == 422
```

scripts/test_api.ps1

```
$ErrorActionPreference = "Stop"

Write-Host "Testing EduGenie..."

$response = Invoke-RestMethod `
    -Uri "http://127.0.0.1:8000/health" `
    -Method Get

$response | ConvertTo-Json
```

9. Security, Limitations & Future Enhancements

Security: keep credentials out of source control; use placeholders in .env.example; validate input; avoid logging secrets. Limitations: no database or production deployment configuration is documented; test execution results were not supplied; the local explainer requires transformers if enabled. Future enhancements include authentication, progress tracking, persistent storage, richer quiz scoring, streaming, observability, CI/CD, accessibility, and analytics.

10. Conclusion

EduGenie combines a FastAPI backend, browser frontend, Gemini 3.8 Flash integration, structured validation, optional local explanation, and API tests into a lightweight educational assistant.